mirai on SLURM

With great power comes great responsibility

Agenda (60 min)

min	
0–5	The problem: it ran fine with 200 resamples…
5–10	**Which tool when?** One job, CPUs, GPU, or optimise
10–17	mirai in 7 minutes
17–32	**Responsibility**: profile *one* task, derive the SLURM request
32–39	Launch right-sized daemons on SLURM, check with `sacct`
39–46	Scaling and task granularity
46–52	**Quarto + SLURM**: where does my code run?
52–56	Checklist, other front-ends
56–60	GPUs: when do they help?

Code: `R/01_…` to `R/09_…`, `quarto-demo/` — run them in order in your Workbench session.

The problem

A hospital group asks…

> Does the new treatment reduce the hazard of readmission-or-death — **at every site**?

- 40 hospital sites, 2,000–10,000 patients each, ~50 lab values per patient
- Per site: Cox model `Surv(time, event) ~ treatment + age + sex + comorbidity`
- Uncertainty: percentile CI from **bootstrap resamples** per site

```r
bootstrap_site <- function(site_df, n_boot = 1000) {
  formula <- survival::Surv(time, event) ~ treatment + age + sex + comorbidity
  log_hr <- vapply(seq_len(n_boot), function(b) {
    idx <- sample.int(nrow(site_df), replace = TRUE)
    stats::coef(survival::coxph(formula, data = site_df[idx, ]))[["treatmentnew"]]
  }, numeric(1))
  data.frame(site = site_df$site[1], hr = exp(median(log_hr)), ...)
}
```

`R/01_simulate_cohort.R`, `R/functions.R`

It worked with 200 resamples…

```
system.time(bootstrap_site(one_site, n_boot = 200))   # the test run
```

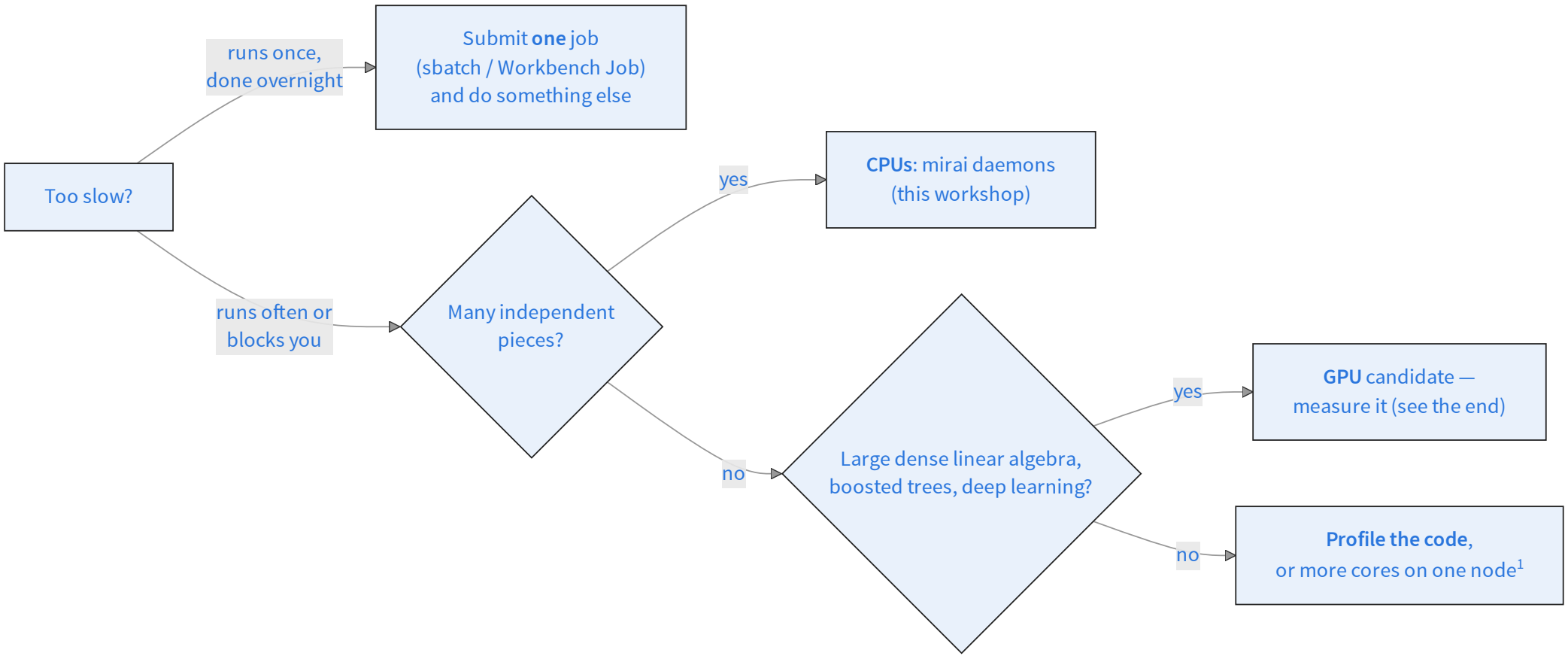
```
Test run: 200 resamples of one site (7589 patients): 4.1 s
Production: 5000 resamples x 40 sites: ~45 minutes (0.8 hours) on one core
```

- Fine for testing; the real analysis needs **25× more resamples × 40 sites** (and the next cohort is bigger)
- Every resample is independent → **embarrassingly parallel**
- One task = **100 resamples of one site** → 5,000 resamples = 2,000 tasks of the same kind

`R/02_serial_baseline.R`

Which tool when?

What does this job need?



The cheapest speed-up is often **not** parallel. Every branch starts with **one measured task**.

[1] Alternatively, explore package-internal parallelisation features

Optimise, parallelise, or let it run?

Let it run when it runs once and finishes overnight — your time costs more than CPU time

Parallelise when there are many independent pieces and you'll rerun it (new data, new `n_boot`, reviewers…)

Optimise first when one piece is slow and you'll run it thousands of times

Our own task, profiled with `Rprof()` / profvis:

- 42% of the time in `[.data.frame`: every resample copies **all 58 columns**, the model uses 5
- Keep 7 columns → **1.65× faster**, same hazard ratios, 8× less data to ship
- That's 1.65× on *every* node you'll ever use — for free

mirai in 7 minutes

Daemons + map

```
library(mirai)
daemons(8)                                  # 8 persistent R workers

site_list <- split(cohort, cohort$site)
everywhere({}, site_list = site_list,       # shared data: once per daemon
           bootstrap_site = bootstrap_site)
tasks <- make_chunks(site_list, n_boot = 5000, chunk = 100)   # 2,000 rows: site, n_boot
draws <- mirai_map(tasks, bootstrap_chunk)[.progress]         # one row = one task
res   <- combine_chunks(tasks, draws, site_list)              # one row per site

daemons(0)                                  # always clean up
```

- Daemons start **empty**: no packages, no globals
  - packages: namespace-qualify inside the function (`bootstrap_site()` calls `survival::coxph()`), or `everywhere(library(survival))`
  - objects: pass them per task (`tasks` columns), via `.args`, or **once per daemon** with `everywhere()`
- Dispatcher hands tasks out FIFO as daemons become free
- `daemons(seed = 42)` → reproducible RNG regardless of how many daemons

Same code, different place

```
# laptop
daemons(8)

# SLURM: each daemon is one sbatch job that dials back to this session
daemons(8, url = host_url(), remote = cluster_config(
  command = "sbatch",
  options = "#SBATCH --mem=???\n#SBATCH --cpus-per-task=???\n#SBATCH --time=???"
))
```

The `mirai_map()` call does not change.

But what goes into `???`

With great power comes great responsibility

Why it matters

- `daemons(200, remote = cluster_config())` is one line — and 200 jobs in the queue
- Ask for **too much**: idle cores and RAM blocked for your colleagues, longer queue waits for *you* (the scheduler has to find bigger holes)
- Ask for **too little**: OOM-killed daemons, `TIMEOUT`, nodes oversubscribed
- Neither shows up as an R error you'd notice at once

Rule: measure one task before you scale to N.

Profile one task, the way it will run

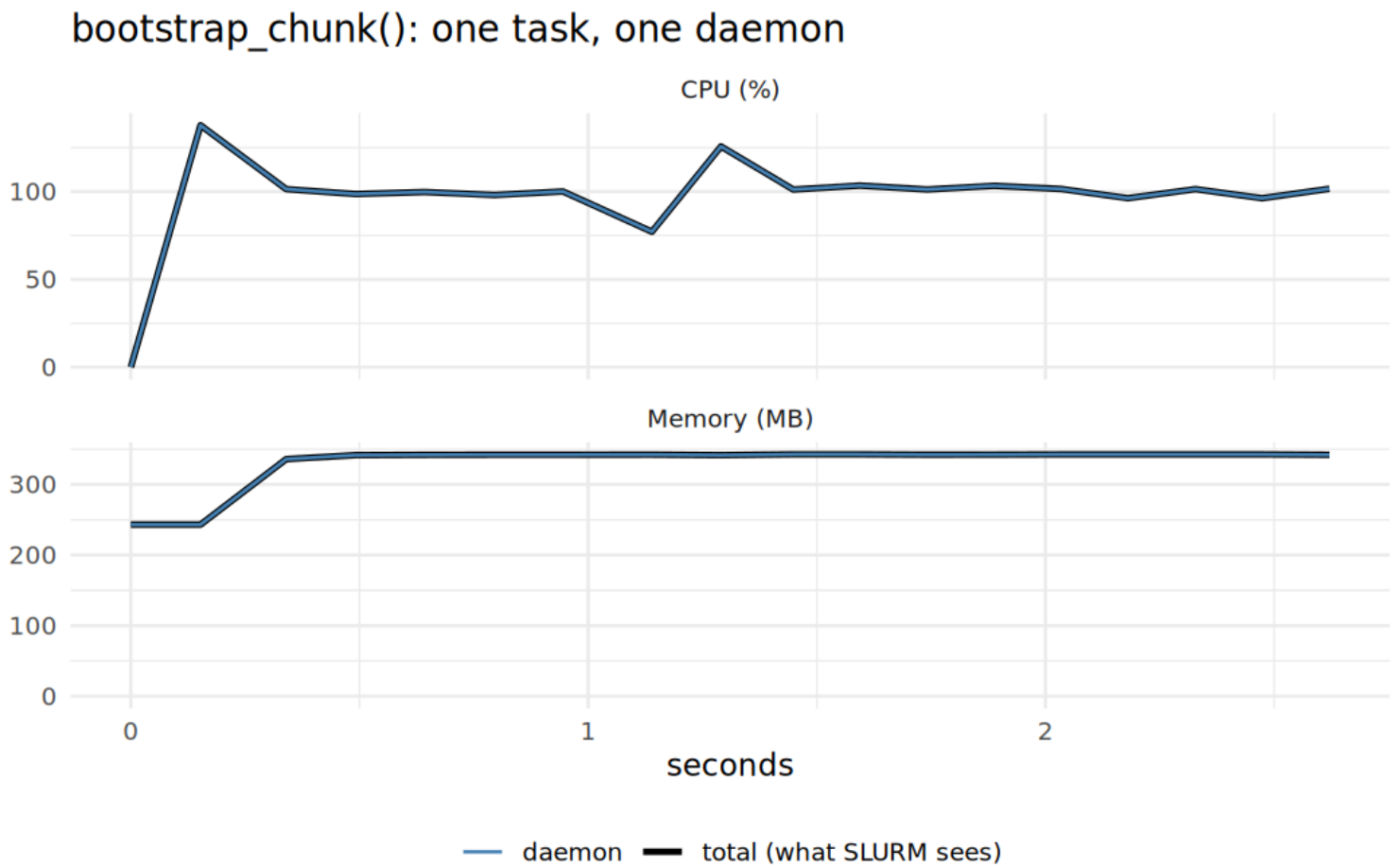
```
profile_task <- function(fun, ..., interval = 0.1) {
  mirai::daemons(1, .compute = "profile")              # ONE fresh local daemon
  on.exit(mirai::daemons(0, .compute = "profile"))
  pid <- mirai::mirai(Sys.getpid(), .compute = "profile")[]
  res <- syrup::syrup(                                 # snapshot `ps` every 0.1s
    mirai::mirai(do.call(fun, args), .args = list(fun = fun, args = list(...)),
                 .compute = "profile")[],
    interval = interval)
  tidy_syrup(res, pid = pid)                           # keep daemon + its children
}

prof <- profile_task(bootstrap_chunk, site = "site_31", n_boot = 100,   # ONE task
                     helpers = list(site_list = site_list, bootstrap_site = bootstrap_site))
```

- Same code path as on the cluster (serialisation, fresh process, shared data on the daemon)
- Profile the **largest** task — memory must fit the worst case
- Fixed task size → **one profile is valid for any `n_boot`**: more resamples = more tasks, not bigger ones
- syrup sees *all* R processes on the machine → filter by PID

`R/03_profile_one_task.R`

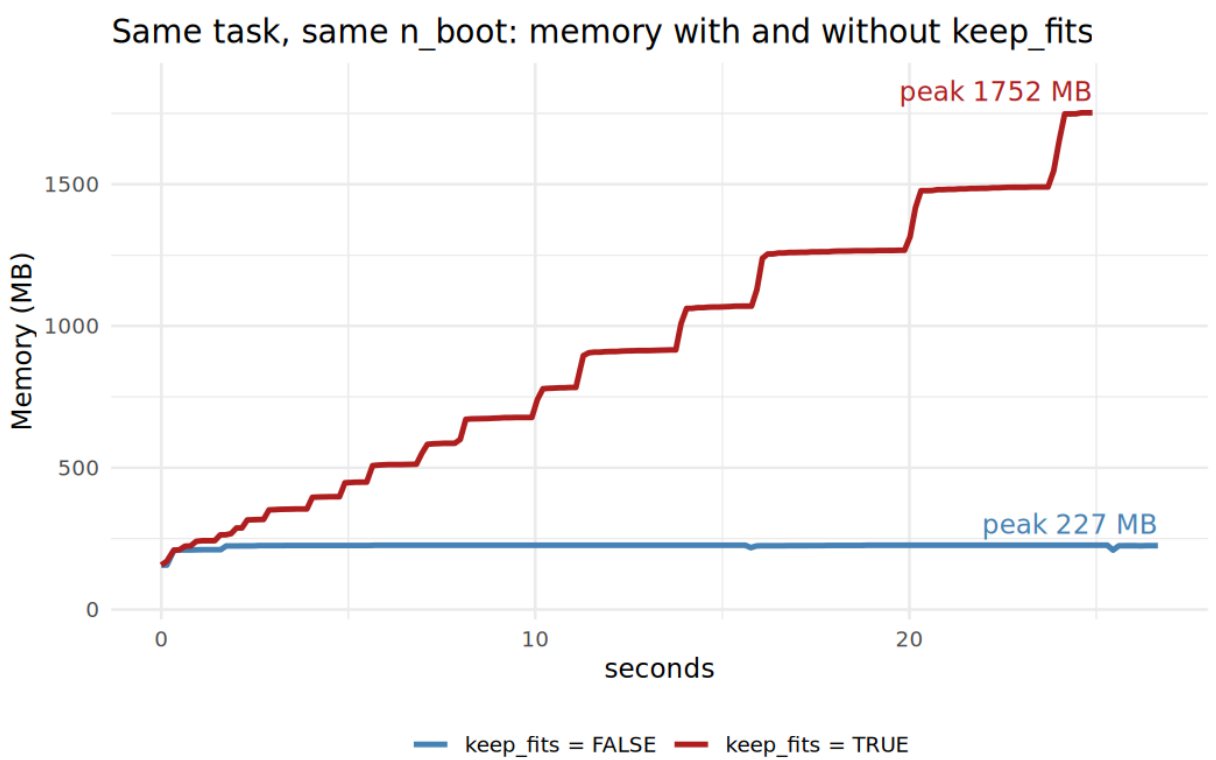
The well-behaved task



One chunk (100 resamples, largest site): 2.7 s, flat ~375 MB (incl. the shared `site_list`).
CPU a flat 100% → **1 core, small memory**

Gotcha 1: memory that grows with the work

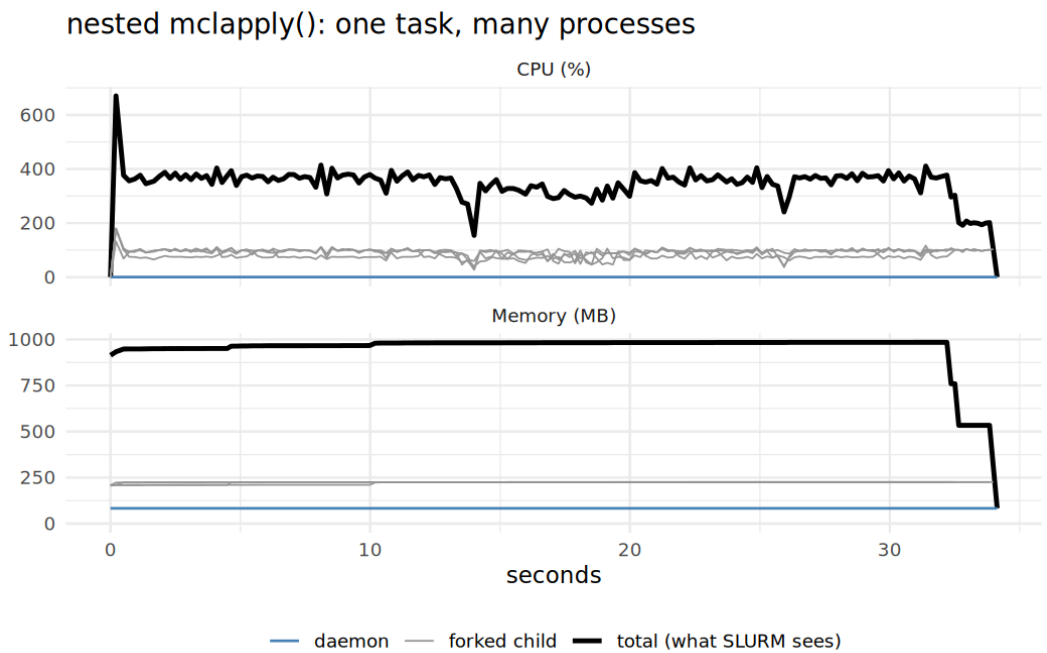
```
profile_task(bootstrap_site, site_df = largest_site, n_boot = 1000, keep_fits = TRUE)
```



- Same task, same 1,000 resamples
- Flat at ~230 MB vs. a ramp to ~1.75 GB (~1.5 MB per kept fit)
- Memory scales with resamples **per task**: chunks of 100 bound it, one 5,000-resample task would need ≈ 8 GB
- **Fix the code**, don't just ask for more RAM

Gotcha 2: hidden parallelism

```
bootstrap_site(..., n_cores = parallel::detectCores())   # a "helpful" colleague
```



`detectCores()` on a compute node = **all cores of the node**, not your allocation. 8 daemons × 64 forks.

In a 1-CPU job on our cluster: `parallel::detectCores()` → **4**, `parallelly::availableCores()` → **1**.

Gotcha 2: what to do

- CPU % well above 100 → the task uses more than one core
- **Never size parallelism with `parallel::detectCores()`**: it counts the hardware, not what you are allowed to use
- Either request `--cpus-per-task=k` **and** respect it:

```
n_cores <- parallelly::availableCores()   # honours SLURM_CPUS_PER_TASK, cgroups, CPU affinity, ...
```

- Or, for embarrassingly parallel work: keep tasks single-threaded and let mirai parallelise
- Same story for multithreaded BLAS / OpenMP / data.table → set `OMP_NUM_THREADS` (on our cluster, openblas-pthread made the “well-behaved” task use ~350% CPU for **zero** speed-up)
- Caveat: forked children share memory pages — summing RSS over-counts

Gotcha 3: shipping more than you need

```
# whole cohort to every task, filter inside
mirai_map(site_ids, function(id) bootstrap_site(cohort[cohort$site == id, ]),
          .args = list(cohort = cohort))
```

	serialised per task	peak RSS
whole cohort	~90 MB	~375 MB
one site (`split()` first)	~4 MB	~230 MB

×40 tasks, over the network, held in every daemon.

Hidden passenger: a function defined *inside* another function ships that function's environment with every task. In our granularity test: 900 KB per task × 40,000 tasks ≈ 35 GB → session killed. Define task functions at top level.

From measurements to #SBATCH

```r
p <- summarise_profile(prof_good)   # wall_s, peak_rss_mb, mean_cpu_pct, ...

resources <- list(
  cpus   = max(1, round(p$mean_cpu_pct / 100)),
  mem_mb = ceiling(p$peak_rss_mb * 1.5 / 256) * 256,          # 1.5x headroom
  time_s = rate * N_BOOT * sum(site_sizes) / N_DAEMONS * 2 + 300   # total work, 2x + start-up
)
cat(slurm_options(resources, partition = "compute"))
```

```
#SBATCH --job-name=mirai-coxboot
#SBATCH --partition=compute
#SBATCH --cpus-per-task=1
#SBATCH --mem=768M
#SBATCH --time=0-00:16:58
#SBATCH --output=logs/%x-%j.out
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

Limits apply **per daemon** (one sbatch job each).

Memory per core: match the node shape

Node ratio = RAM ÷ cores, e.g. 8 cores, 32 GB → **4 GB per core**

- Task needs ≤ the ratio → SLURM can fill every core of a node
- Task needs more → memory runs out before cores do: the remaining cores sit idle *and* are blocked for others

Our nodes: 4 cores, 14.8 GB → **3.7 GB per core** (`DefMemPerCPU=3700`)

task	`--mem`	daemons per node	idle cores
`bootstrap_chunk()` (100 resamples)	768 MB	4	0
`keep_fits = TRUE`, 5,000 resamples	~12 GB	1	3

```
sinfo -N -o "%N %c %m"                    # cores and memory (MB) per node
scontrol show config | grep DefMemPerCPU
```

Launching on SLURM

Right-sized daemons

```
daemons(
  n        = 8,
  url      = host_url(),                        # compute nodes dial back here
  remote   = cluster_config(command = "sbatch",
                            options = slurm_options(resources, partition = "compute"),
                            rscript = RSCRIPT),
  seed     = 42,
  idletime = 5 * 60 * 1000,                     # idle daemon exits -> frees allocation
  walltime = (resources$time_s - 120) * 1000    # stop before SLURM kills us
)
wait_for_daemons(8)                             # jobs may sit in the queue

everywhere({}, site_list = site_list, bootstrap_site = bootstrap_site)
draws <- mirai_map(make_chunks(site_list, 5000, 100), bootstrap_chunk)[.progress]
daemons(0)
```

- Sort tasks **largest first** → a big chunk doesn't finish last on its own
- Shared data via `everywhere()`: 2,000 tasks each carrying their site's data would queue ~4 GB in *your* session
- Compute nodes need network access to the Workbench session and the same R packages

`R/04_launch_slurm.R`

Alternative: the Workbench Launcher

```
daemons(
  n      = 8,
  url    = host_url(),
  remote = http_config(cpus = resources$cpus, memory = resources$mem_mb),
  seed   = 42
)
```

- Workbench submits the jobs for you (admin-configured cluster / resource profiles)
- Same lesson: pass the **measured** `cpus` / `memory`, not the biggest profile

Close the loop: requested vs. used

```
sacct --name=mirai-coxboot -S today --units=M \
      --format=JobID,State,ReqCPUS,ReqMem,Timelimit,Elapsed,TotalCPU,MaxRSS
seff <jobid>
```

- Usage (`TotalCPU`, `MaxRSS`) is on the `.batch` step line, not the job line
- `MaxRSS` ≪ `ReqMem` → lower `--mem` next time
- `TotalCPU` ≈ `Elapsed × ReqCPUS` → the cores were actually used
- `OUT_OF_MEMORY` / `TIMEOUT` → back to profiling

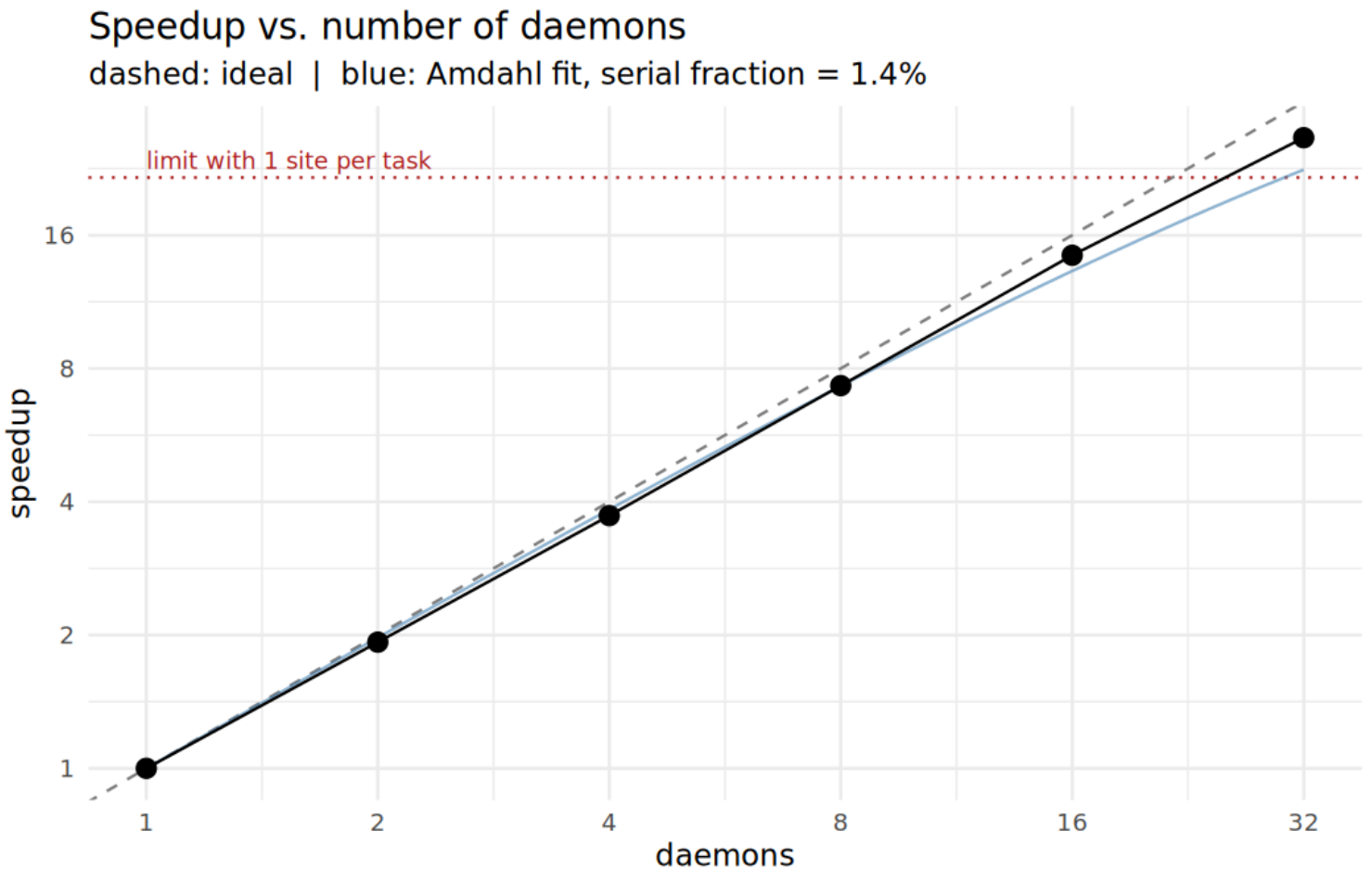
Scaling

1, 2, 4, 8, … daemons

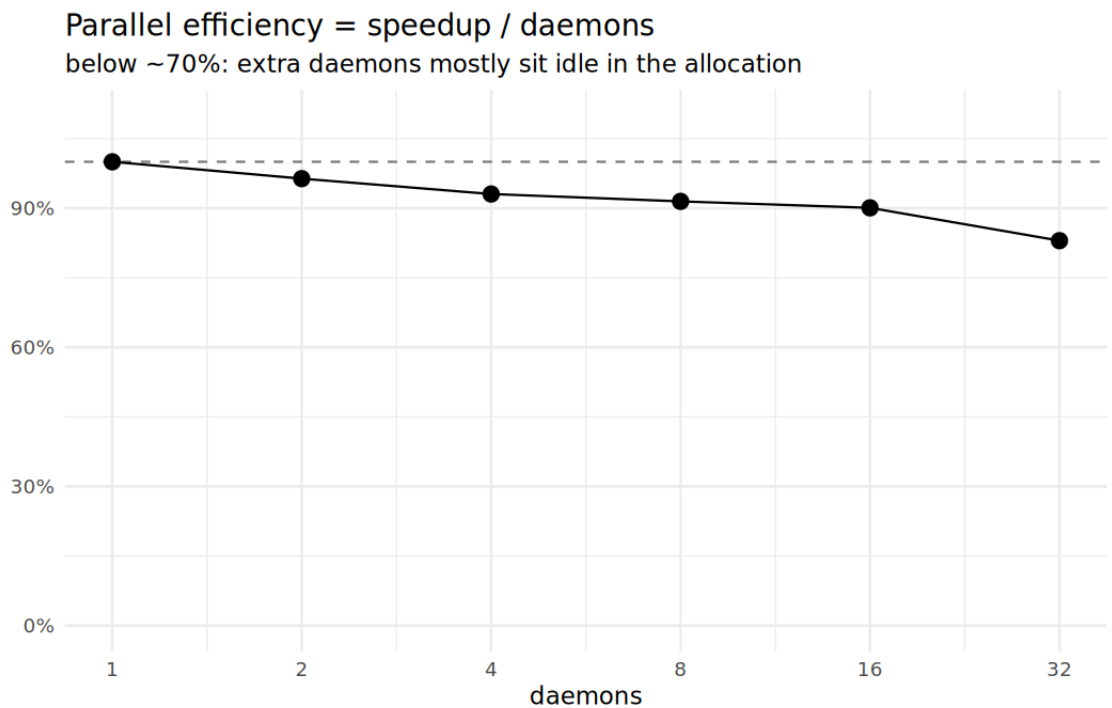
```
for (n in c(1, 2, 4, 8, 16, 32)) {
  start_daemons(n, resources, ...)                        # queue time measured separately
  t <- system.time(mirai_map(make_chunks(site_list, 1000, 100), bootstrap_chunk)[])
  daemons(0)
}
```

R/05_scaling.R, R/06_plot_scaling.R

Speedup


Speedup vs. number of daemons
dashed: ideal | blue: Amdahl fit, serial fraction = 1.4%
limit with 1 site per task
16
8
4
2
1
speedup
1
2
4
8
16
32
daemons

Efficiency



- One site per task would hit a floor: never faster than the **largest site** (dotted line) → chunks avoid it
- Beyond ~70% efficiency loss, extra daemons are mostly idle allocations
- On a busy cluster, **queue time** for many jobs can outweigh the compute gain

Why 100 resamples per task?

Same work (40 sites × 1,000 resamples), different task sizes. Efficiency vs. 1 daemon:

one task =	tasks	8	24	30
1 site	40	87%	80%	**65%**
100 resamples	400	90%	88%	**88%**
10 resamples	4,000	89%	87%	87%
1 resample (~13 ms)	40,000	86%	84%	84%

```
everywhere({}, site_list = site_list)   # ship shared dat
```

- **Too coarse**: few, unequal tasks → the largest one is the floor (27 s at 24 *and* 30 daemons)
- Split big tasks into similar-sized chunks, combine afterwards → 65% → 88%
- **Too fine**: every task pays dispatch + result overhead (~0.1 ms in mirai). 13 ms tasks still lose ~4%; sub-ms tasks → group them into chunks
- Rule of thumb: **≥ 10× more tasks than daemons**, each ≥ ~0.1 s → our choice: 100 resamples
- `R/08_granularity.R`

Quarto + SLURM

Pattern: compute in a job, render in seconds

```
sbatch fit_job.sh              # R/04_launch_slurm.R: 2,000 tasks on SLURM daemons -> saveRDS()
quarto render report.qmd       # the .qmd only does readRDS(): renders anywhere, in seconds
```

```
hr <- readRDS("results/site_hazard_ratios.rds")      # in report.qmd
```

- Rerendering for a typo shouldn't rerun 45 CPU-minutes
- In a Quarto project: `execute: freeze: auto` keeps computed results until the code changes; `cache: true` per chunk
- Or render the whole document as a job: `sbatch -c 4 --mem=4G --wrap 'quarto render report.qmd'`
  - From the RStudio IDE: the quarto-workbenchjobs addin renders the open document as a Workbench job

Where does my Quarto code run?

8 tasks with `future_map()`, `plan(multisession, workers = availableCores())` · measured with `quarto-demo/`

started as	runs in	gets	workers	time
console (“Run All”)	session job	4 CPU, 4 GB	4	4.8 s
`quarto render` in session	**same** session job	4 CPU, 4 GB	4	4.4 s
`sbatch -c 4` render	its own job	4 CPU, 4 GB	4	4.3 s
`sbatch -c 1` render	its own job	**1 CPU**, 2 GB	**1**	8.4 s
session render + daemons	session + 4 daemon jobs	4 CPU, 4 GB	4	4.1 s
`sbatch -c 1` render + daemons	1-CPU job + 4 daemon jobs	1 CPU, 2 GB	4	4.1 s

- Rendering starts a new R process **inside the allocation that started it** — `detectCores()` said 4 every time
- Big parallel work in a report: keep the render job small and send the work to daemons (last row)

Wrap-up

Checklist before `daemons()`

Before you launch

0. Decide: one job overnight, many CPUs, or a GPU?
1. Profile **one** task (the largest); fixed task size
2. Memory flat? CPU ≈ 100%? Else fix the code
3. Ship only what tasks need; task functions at top level
4. Return only what post-processing needs
5. `--mem`, `--cpus-per-task`, `--time` with headroom; memory per core ≤ node RAM ÷ cores

When you run

6. `idletime` / `walltime`; `daemons(0)` when done
7. Check `sacct` / `seff`, then refine
8. Pick `n` from the scaling curve
9. ≥ 10× more tasks than daemons, each ≥ ~0.1 s

Same daemons, other front-ends

Here one task = one whole site (40 **unequal** tasks):

```
start_daemons(8, resources, ...)    # = daemons(8, url = host_url(), remote = cluster_config(...))

purrr::map(site_list, purrr::in_parallel(                    # purrr >= 1.1
  \(d) bootstrap_site(d, n_boot = 5000), bootstrap_site = bootstrap_site))

future::plan(future.mirai::mirai_cluster)                    # furrr, future.apply, doFuture, ...
furrr::future_map(site_list, bootstrap_site, n_boot = 5000,
                  .options = furrr::furrr_options(seed = TRUE))

cl <- make_cluster(8, url = host_url())                      # base parallel: parLapply(), ...
launch_remote(cl, remote = cluster_config(...))             # one sbatch job per node
```

- The **daemons** are what SLURM sizes → profiling and `#SBATCH` lessons carry over unchanged
- `in_parallel(...)` = ship only what you need, just like `.args`
- `make_cluster(8, remote = cluster_config(...))` infers *1* node from the config → `parLapply()` runs serially (~6× slower here). Check `length(cl)`!
- `R/07_other_frontends.R`

Static chunks vs. one task at a time

40 sites, 8 daemons, same workers:

front-end	default	one at a time
`mirai_map()` / purrr	**~18 s**	(default)
furrr	28 s	**18 s**
`parLapply()`	27 s	**17 s**
crew.cluster		21 s (incl. starting workers)

```
furrr_options(scheduling = Inf)
parLapply(cl, x, f, chunk.size = 1)   # or clusterApply
```

- furrr and `parLapply()` split the work into **one chunk per worker, up front**
- Sites sorted largest first → chunk 1 gets the 5 biggest (43k rows), chunk 8 the 5 smallest (10k rows)
- 7 workers finish early and wait — idle, but still allocated
- mirai / purrr hand out **one task at a time** → right for unequal tasks

crew / crew.cluster (and targets)

```
controller <- crew.cluster::crew_controller_slurm(
  workers = 8, seconds_idle = 300,
  options_cluster = crew.cluster::crew_options_slurm(
    partition = "compute", cpus_per_task = 1,
    memory_gigabytes_per_cpu = 0.5, time_minutes = 30
  )
)
targets::tar_option_set(controller = controller)
```

- crew launches and **auto-scales** its own mirai daemons as one SLURM **job array**
- … but the per-worker cpus / memory / time still come from **your profile** (from 03: 1 CPU, 768M per worker)
- `memory_gigabytes_per_cpu` → think node shape (≤ 3.7 GB per core here)

GPUs: when do they help?

GPUs: what they buy *us*

55 predictors, 2M patients (Stan: 500k, 1 chain) · CPU: m7a · GPU: NVIDIA T4 · same answers in every case

	CPU	GPU	gain
xgboost, 200 trees	13.3 s	3.8 s	**3.5×**
… via tidymodels `boost_tree()`	19.2 s	9.0 s	2.1×
8192^2 matrix multiply	3.6 s	0.29 s	**12×**
logistic GLM: `glm()` vs. cuda.ml	13.4 s	9.2 s	1.5×
… own torch IRLS, fp64 / fp32	11.6 s	4.3 / 0.95 s	3× / 14×
Bayesian logistic GLM: CmdStan · brms (OpenCL)	373 · 389 s	63 · 34 s	**6× · 11×**
`coxph()` 2M rows · `glmer()` 500k rows	6.3 · 7.5 s	no GPU path	—

`R/09_gpu_benchmarks.R`, `R/10_cmdstan_opencl.R`

When is a GPU worth it?

Worth trying

- Boosted trees on large data: `device = "cuda"` (one `set_engine()` argument in tidymodels)
- Deep learning: torch, tensorflow/keras
- Large dense linear algebra
- cuda.ml: GPU engines for `logistic_reg()`, `rand_forest()`, SVMs
- Stan / brms GLM likelihoods (`bernoulli_logit_glm`, `normal_id_glm`, …): `brm(..., opencl = opencl(c(0, 0)))`

Don’t bother

- Cox, lme4 / glmmTMB: no GPU path
- Anything that fits in seconds
- Many small fits (one Cox fit here: 0.04 s) → **CPUs in parallel**
- brms / Stan models outside those functions: OpenCL speeds up “only some Stan functions” (brms docs)

Before you switch to a GPU

- **Precision**: a T4 is fast in fp32, slow in fp64 — yet Stan's fp64 GLM still ran 6–11× faster. Measure, don't guess; check the answers
- **Setup cost**: GPU xgboost isn't on CRAN, torch needs its CUDA build, cuda.ml downloads 1.6 GB
- **The CPU side still matters**: our GPU node has 2 vCPU — tidymodels' data prep took longer than the GPU fit
- **brms gotcha**: after a CPU fit of the same model, add `stan_model_args = list(force_recompile = TRUE)` — or the cached CPU binary is reused and the chain fails
- **Same responsibility**: `#SBATCH --gres=gpu:1`, profile with `nvidia-smi` — GPU memory is the new `--mem`

Resources

- mirai: https://mirai.r-lib.org
- syrup: https://simonpcouch.github.io/syrup/
- purrr `in_parallel()`: https://purrr.tidyverse.org/reference/in_parallel.html
- future.mirai / furrr: https://future.mirai.futureverse.org, https://furrr.futureverse.org
- crew.cluster: https://wlandau.github.io/crew.cluster/
- GPU: xgboost GPU builds, torch, cuda.ml
- Workshop code: this repository, `R/01_…` – `R/09_…`, `quarto-demo/`, `slurm/`

Appendix

The simulated cohort

`R/01_simulate_cohort.R` calls this (from `R/functions.R`) and saves `data/cohort.rds`:

```r
cohort <- simulate_cohort(n_sites = 40)   # seed = 2026 -> same data every time
```

```r
simulate_cohort <- function(n_sites = 40, min_patients = 2000, max_patients = 10000,
                            n_biomarkers = 50, seed = 2026) {
  set.seed(seed)
  site_sizes <- round(stats::runif(n_sites, min_patients, max_patients))
  n <- sum(site_sizes)

  site       <- rep(sprintf("site_%02d", seq_len(n_sites)), times = site_sizes)
  site_idx   <- rep(seq_len(n_sites), times = site_sizes)
  age        <- round(stats::rnorm(n, 65, 12))
  sex        <- factor(sample(c("F", "M"), n, replace = TRUE))
  comorbidity <- stats::rpois(n, 1.5)                 # Charlson-like index
  treatment  <- factor(stats::rbinom(n, 1, 0.5), labels = c("standard", "new"))

  site_log_hr   <- stats::rnorm(n_sites, log(0.75), 0.15)   # true per-site effect
  site_baseline <- stats::rnorm(n_sites, log(0.02), 0.3)

  lp <- site_baseline[site_idx] +
    site_log_hr[site_idx] * (treatment == "new") +
    0.03 * (age - 65) + 0.2 * (sex == "M") + 0.25 * comorbidity
  event_time  <- stats::rexp(n, rate = exp(lp))
  censor_time <- stats::runif(n, 0, 60)                # months of follow-up

  cohort <- data.frame(
    patient_id  = seq_len(n),
    site        = site,
```